

```

stegoController.js-const fs = require("fs");
const crypto = require("crypto");
const path = require("path");
const sharp = require("sharp");

// AES encryption
const encryptMessage = (message, key) => {
  const iv = crypto.randomBytes(16);
  const cipher = crypto.createCipheriv("aes-256-cbc",
  crypto.createHash("sha256").update(key).digest(), iv);
  let encryptedMessage = cipher.update(message, "utf8",
  "hex");
  encryptedMessage += cipher.final("hex");
  return { iv: iv.toString("hex"), encryptedMessage };
};

// AES decryption
const decryptMessage = (encryptedMessage, key, iv) => {
  const decipher = crypto.createDecipheriv("aes-256-cbc",
  crypto.createHash("sha256").update(key).digest(),
  Buffer.from(iv, "hex"));
  let decryptedMessage =
  decipher.update(encryptedMessage, "hex", "utf8");
  decryptedMessage += decipher.final("utf8");
  return decryptedMessage;
};

// Encode message into image
const encodeMessageInImage = async (imagePath,
  encryptedData) => {
  const { iv, encryptedMessage } = encryptedData;
  const payload =
  Buffer.from(`${iv}:${encryptedMessage}`).toString("base64"); // Use Base64 encoding
  const payloadBits = payload.split("").map((char) =>
  char.charCodeAt(0).toString(2).padStart(8, "0")).join("");

  const image = sharp(imagePath);
  const { data, info } = await image.raw().toBuffer({
  resolveWithObject: true });

  if (payloadBits.length > data.length) {
    throw new Error("Message is too large to fit in the
    image.");
  }

  for (let i = 0; i < payloadBits.length; i++) {
    data[i] = (data[i] & 0xFE) | parseInt(payloadBits[i], 2);
  }

  const encodedImagePath = path.join(__dirname,
  "../uploads/encoded", encoded_${Date.now()}.png);
  await sharp(data, { raw: { width: info.width, height:
  info.height, channels: info.channels } })
    .toFormat("png")
    .toFile(encodedImagePath);

  return encodedImagePath;
};

```

```

// Decode message from image
const decodeMessageFromImage = async (imagePath,
key) => {
  const image = sharp(imagePath);
  const { data } = await image.raw().toBuffer({
    resolveWithObject: true });

  const payloadBits = [];
  for (let i = 0; i < data.length; i++) {
    payloadBits.push((data[i] & 0x01).toString());
  }

  const binaryString = payloadBits.join("");
  const byteArray = binaryString.match(/.{8}/g) || []; //
  Ensure no null values
  const payload = Buffer.from(byteArray.map((byte) =>
String.fromCharCode(parseInt(byte, 2))).join(""),
"base64").toString();

  const [iv, encryptedMessage] = payload.split(":");
  if (!iv || !encryptedMessage) {
    throw new Error("Failed to decode the message. Ensure
the correct key was used.");
  }

  return decryptMessage(encryptedMessage, key, iv);
};

// Encode controller
const encode = async (req, res) => {
  const { message, key } = req.body;
  const imagePath = req.file.path;

  try {
    const encodedImagePath = await
encodeMessageInImage(imagePath,
encryptMessage(message, key));

    res.json({
      message: "Image encoded successfully!",
      imagePath:
/uploads/encoded/${path.basename(encodedImageP
ath)},
    });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// Decode controller
const decode = async (req, res) => {
  const { key } = req.body;
  const imagePath = req.file.path;

  try {
    const decodedMessage = await
decodeMessageFromImage(imagePath, key);
    res.json({ decodedMessage });
  } catch (error) {

```

```

    res.status(500).json({ error: error.message });
  }
};

module.exports = { encode, decode }; stegoRoutes.js-
const express = require("express");
const router = express.Router();
const { encode, decode } =
require("../controllers/stegoController");

const stegoRoutes = (upload) => {
  router.post("/encode", upload.single("image"), encode);
  router.post("/decode", upload.single("image"), decode);
  return router;
};

module.exports = stegoRoutes;

server.js-const express = require("express");
const multer = require("multer");
const path = require("path");
const cors = require("cors");

const stegoRoutes = require("./routes/stegoRoutes");

const app = express();
const port = 5000;

app.use(cors());
app.use(express.json());
app.use("/uploads", express.static(path.join(__dirname,
"uploads")));

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null,
path.join(__dirname, "uploads/temp")),
  filename: (req, file, cb) => cb(null,
`${Date.now()}_${file.originalname}`),
});

const upload = multer({ storage });

app.use("/api/steganography", stegoRoutes(upload));

app.listen(port, () => {
  console.log(Server running on
http://localhost:${port});
});
which algorithms are used

```

In your digital image steganography application, you're using the following **algorithms** and **libraries**:

Encryption Algorithm

AES-256-CBC (Advanced Encryption Standard - 256-bit, Cipher Block Chaining mode)

Used for securely encrypting and decrypting messages.

- Implemented using Node.js's built-in `crypto` module.
- A SHA-256 hash of the key is used to derive the AES key.
- Initialization Vector (IV) is generated randomly for each encryption.

js



Copy



Edit

```
crypto.createCipheriv("aes-256-cbc",  
crypto.createHash("sha256").update(key).digest(), iv);
```



Image Processing and Steganography Technique

LSB (Least Significant Bit) Steganography

Used to hide the encrypted message inside the image.

- You convert the encrypted message (including IV) to **Base64**, then to a **binary bit string**.
- Each bit is hidden in the **least significant bit** of each image pixel byte (R, G, B, or A channel).
- This is a common and simple form of image steganography.

js



Copy



Edit

```
data[i] = (data[i] & 0xFE) | parseInt(payloadBits[i], 2);
```

This line replaces the least significant bit of each byte with a bit from the payload.



Image Manipulation Library

Sharp

Used for reading raw image buffers and writing images.

- `sharp(imagePath).raw()` extracts pixel data.
- `sharp(data, { raw: ... })` is used to re-encode the manipulated image.



File Upload Handling

Multer (middleware)

Used to handle multipart/form-data (image uploads).



Summary of Algorithms and Techniques Used

Purpose	Algorithm / Method	Library / Module
Message Encryption	AES-256-CBC	crypto
Key Derivation	SHA-256 Hash	crypto
Data Hiding in Image	LSB (Least Significant Bit)	Manual logic
Image Processing	Raw pixel manipulation	sharp
File Upload Handling	-	multer

Let me know if you'd like to **upgrade the encryption** (e.g., add password salting) or **support more complex steganography methods** (e.g., DCT-based for JPEGs).



for data hiding and encryption and data description

Great! Here's a **detailed breakdown** of the techniques used in your code for **data hiding, encryption, and data extraction** (decryption):

 1. Data Encryption (Before Hiding in Image)

 Technique Used: AES-256-CBC

- AES stands for Advanced Encryption Standard, a symmetric block cipher.
- 256-bit key size makes it very secure.
- CBC mode (Cipher Block Chaining) adds randomness via an IV (Initialization Vector).

 How it's done in your code:

js  Copy  Edit

```
const iv = crypto.randomBytes(16); // Random IV for CBC mode
const cipher = crypto.createCipheriv("aes-256-cbc",
crypto.createHash("sha256").update(key).digest(), iv);
```

- Message is encrypted using a key derived from SHA-256 hash of the user input.
- Final payload:

plaintext  Copy  Edit

```
iv:encryptedMessage
```

This is Base64-encoded and hidden in the image.

 2. Data Hiding (Steganography)

🌟 Technique Used: LSB (Least Significant Bit) Manipulation

- Each bit of the message is embedded into the last bit (least significant bit) of each pixel byte (R, G, B channels).
- This causes minimal distortion in image quality (invisible to the human eye).

🔧 How it's done in your code:

js

 Copy Edit

```
data[i] = (data[i] & 0xFE) | parseInt(payloadBits[i], 2);
```

- `data[i] & 0xFE` : clears the last bit of the byte.
- `| parseInt(...)` : inserts the bit from the message.
- The encoded message bits are spread sequentially across the raw pixel data of the image.

🔒 3. Data Extraction & Decryption (After Decoding the Image)

🔍 Step-by-Step Process:

(a) Extracting Bits from Image:

js

 Copy Edit

```
payloadBits.push((data[i] & 0x01).toString());
```

- Extracts the last bit from each byte of image data.
- Reconstructs the original bitstream.

(b) Convert Binary → Base64 → IV:EncryptedMessage

js

 Copy Edit

```
const binaryString = payloadBits.join(""); const byteArray =  
binaryString.match(/.{8}/g); // Group every 8 bits const payload =  
Buffer.from(...).toString();
```

(c) Decryption using AES-256-CBC

js

 Copy Edit

```
const decipher = crypto.createDecipheriv("aes-256-cbc", hashedKey, ivBuffer);
```

- Same key is used to decrypt.
- If the key or image is tampered, decryption fails or returns garbage.

🧠 Conceptual Summary

Step	Technique	Purpose
Encrypt message	AES-256-CBC with SHA-256 key	Secure the message
Encode message	LSB (Least Significant Bit)	Hide the message inside an image
Base64 encoding	Encoding	Convert binary to image-storable format
Decode message	Bitstream → Base64 → Decrypt	Extract and recover original message

If you're building a project report or presentation, I can help you create **diagrams**, **flowcharts**, or **technical writeups** for these sections too. Just let me know!



exmp~~le~~

Sure, here's a **step-by-step example** showing how your image steganography and encryption logic works using a simple message:

 Example Scenario

- **Original Message:** Hello
- **Secret Key (password):** mypassword
- **Carrier Image:** A .png image with enough pixels

 Step 1: AES-256-CBC Encryption

Input:

- Message: Hello
- Key: mypassword

Behind the scenes:

1. Create 256-bit encryption key using SHA-256 hash of "mypassword" .
2. Generate a random 16-byte IV.
3. Encrypt "Hello" using AES-256-CBC.

Output:

Let's say:

- IV: 6abf5e1c2e4a8f07bb4dcbd3770ed41c
- Encrypted Message (hex): b7aa781b93f0fa60d7f3eb93a28a50da

These are concatenated as:

makefile



Copy



Edit

Payload: 6abf5e1c2e4a8f07bb4dcdbd3770ed41c:b7aa781b93f0fa60d7f3eb93a28a50da

 Step 2: Base64 Encode the Payload

This becomes:

yaml



Copy



Edit

Base64 Payload: NmFiZjVlMWMYzTRhOGYwN2JiNGRjYmQzNzcwZWQ0MWM6YjdhYTc4MWI5M2YwZmE2M

 Step 3: LSB Embedding in Image

- 1. Convert each character of Base64 to its binary equivalent.
- 2. Embed each bit into the least significant bit of each image byte.
- 3. Save the modified image as encoded_<timestamp>.png .

So for example:

- Letter N = ASCII 78 = binary 01001110
- You embed these bits into the image one by one.

 Step 4: Extract & Decode Message from Image

- 1. Read LSBs from image pixel bytes.
- 2. Reconstruct binary → Base64 → original encrypted payload.
- 3. Split it into IV and Encrypted Message.

makefile



Copy



Edit

Extracted Payload:
IV = 6abf5e1c2e4a8f07bb4dcdbd3770ed41c
Encrypted = b7aa781b93f0fa60d7f3eb93a28a50da

- 4. Use AES-256-CBC + "mypassword" to decrypt.
- 5.  Result: "Hello"

 Final Result

Operation	Result
Original Message	Hello
Key	mypassword
Encrypted Text	b7aa781b93f0fa60d7f3eb93a28a50da

Operation	Result
Hidden Image	encoded_<timestamp>.png
Decoded Message	Hello

Let me know if you want me to generate a **diagram** or **code walkthrough** of this example — or want to try this with a sample image you have!



use of lsb and aes

Great question, Pratik! Here's a clear explanation of the **use of LSB** and **AES** in your image steganography project — both serve distinct purposes but work **together** to ensure data **confidentiality** and **invisibility**.

AES (Advanced Encryption Standard) – For Confidentiality

◆ **Purpose:**

To **secure** your message *before* hiding it in the image, so even if someone extracts the hidden data, they can't read it without the key.

◆ **Why AES?**

- It's a **symmetric key encryption algorithm** (same key for encryption & decryption).
- **AES-256** is widely used in industries (banks, military) due to its strength.
- In your code, it's used in **CBC mode** with a **random IV** for each message, making each encryption unique even if the message is the same.

◆ **Example:**

Before hiding:

pgsql

Original: Hello

Encrypted with AES → b7aa781b93f0fa60d7f3eb93a28a50da

Copy Edit

If someone opens the image file with a hex viewer or tries to read bits, they'll only see **encrypted gibberish**, not "Hello".

LSB (Least Significant Bit) – For Invisibility

◆ **Purpose:**

To **hide the encrypted message** inside the image by changing the **last bit** of pixel values. This doesn't noticeably change the image visually.

◆ Why LSB?

- Image pixels are made of RGB (and sometimes Alpha) channels.
- Each channel is 8 bits (e.g., R = 11100101)
- Changing the last bit (e.g., to 11100100) doesn't change the color significantly.

◆ What's being hidden?

You're not hiding the plain message, but the **encrypted + base64-encoded** message from AES.

◆ Example:

If you want to hide 01101101 (8 bits), you'll slightly change 8 pixels:

plaintext

Copy

Edit

Original bytes: 11100101 10011100 11111111 ... Hidden bits: ↓ ↓ ↓ Modified bytes: 11100100 10011101 11111111 ...

Human eye sees **no difference**, but you've secretly stored your data.

🧠 Together – Why Combine AES + LSB?

Feature	LSB Only	AES + LSB (your method)
Security	❌ Anyone can extract hidden data	✅ Extracted data is encrypted (useless w/o key)
Stealth (invisible)	✅ Looks like a normal image	✅ Looks like a normal image
Tamper Protection	❌ Not protected from brute-force	✅ AES makes brute-force impractical

✅ Summary

Technique	Role	Your Code
AES-256-CBC	Encrypts the message using password (for security)	encryptMessage() and decryptMessage()
LSB	Hides the encrypted data inside image pixels (for stealth)	encodeMessageInImage() and decodeMessageFromImage()

If you're including this in a presentation, I can create a clean **diagram** or **slide-style explanation** too! Want me to do that?



here encoding and decoding which algorithms or technique are used??

Awesome! Let's break it down super clearly:

In your project, **encoding** and **decoding** involve **two main algorithms/techniques** working together:

 Encoding and Decoding – Techniques Used

Process	Technique/Algorithm	Purpose
Encoding	1. AES-256-CBC	Encrypts the message for security
	2. LSB (Least Significant Bit)	Hides encrypted message in image
Decoding	1. LSB	Extracts hidden data from image
	2. AES-256-CBC	Decrypts to get original message

 1. AES-256-CBC (Encryption + Decryption)

- AES (Advanced Encryption Standard) with 256-bit key and CBC (Cipher Block Chaining) mode.
- Encrypts your message **before** embedding it in the image.
- Decrypts the extracted message **after** it's pulled from the image.

♦ Example in your code:

js

Copy

Edit

```
const cipher = crypto.createCipheriv("aes-256-cbc", key, iv); // Encryption
const decipher = crypto.createDecipheriv("aes-256-cbc", key, iv); //
Decryption
```

 2. LSB (Least Significant Bit) Technique

- Hides the **encrypted message** inside the image by modifying the least significant bits of pixel values.
- Doesn't visibly change the image.

♦ Example in your code:

js

Copy

Edit

```
// Embed bits in encoding data[i] = (data[i] & 0xFE) |
parseInt(payloadBits[i], 2); // Extract bits in decoding
```

```
payloadBits.push((data[i] & 0x01).toString());
```

Summary (Flow)

+ Encoding

1. User enters: "Hello" and a key "abc123"
2.  Encrypt "Hello" with AES → Encrypted string
3.  Convert encrypted string to binary → Embed using **LSB** in image pixels
4. Result: A normal-looking **encoded image**

— Decoding

1. Upload the encoded image + password
2.  Extract LSB bits from image → get encrypted string
3.  Decrypt using AES → Original message "Hello"

✓ Final Answer:

✓ Encoding uses:

- AES-256-CBC for encryption
- LSB (Least Significant Bit) for hiding data in the image

✓ Decoding uses:

- LSB to extract hidden data
- AES-256-CBC to decrypt it back to the original message